

Chapter 11

SWEB: putting it together

SWEB is the course’s teaching kernel: small enough to read, real enough to boot on x86-64 hardware (in QEMU). This last chapter is a map — where things are, how the machine gets from power-on to the shell, where each idea of chapters 1–10 lives in the code — and a guide to the A1 work: what changes, and how to test it so that it survives the tenth run. **module 11’s** worksheet walks through all of it with questions.

11.1 The source tree

Path	Contents
common/source/kernel/	main.cpp (startup), Thread, UserProcess, Scheduler, Syscall, Loader, ProcessRegistry, locks (Lock, Mutex, SpinLock, Condition), CleanupThread, IdleThread
common/source/mm/	PageManager (frames), KernelMemoryManager (kernel heap), PageFaultHandler
common/source/fs/, console/	VFS, minix file system, terminal, kprintf and debug()
common/include/kernel/	headers, syscall-definitions.h (numbers, shared with the libc), user_progs.h
common/include/console/debug.h	debug categories (switch output on/off per category)
arch/x86/64/	page tables (ArchMemory), registers and context switch (ArchThreads, arch_interrupts.S), the IDT and all interrupt handlers (InterruptUtils.cpp), offsets.h (USER_BREAK)
userspace/libc/	SWEB’s own small C library: write.c, pthread.c (stubs), nonstd.c (_start)
userspace/tests/	user programs; every x.c becomes x.sweb on the disk image

11.2 From power-on to the shell

1. GRUB loads the kernel (the menu has no timeout — press Enter); early assembly sets up paging and jumps to startup().

2. **startup()** creates the console, the scheduler (with the cleanup and idle threads), mounts the root file system and adds the **ProcessRegistry** thread; then it enables interrupts and yields — from now on the scheduler runs everything.
3. **ProcessRegistry::Run** mounts the user-program partition at `/usr` and creates a **UserProcess** for each entry of **user_progs** — by default only `/usr/shell.sweb`.
4. The **UserProcess** constructor opens the binary, creates a **Loader** (reads the ELF headers), maps one stack page below **USER_BREAK**, sets the user registers (**rip** = ELF entry = **_start**, **rsp** = **USER_BREAK** - 8) and the address space; the scheduler switches to it in user mode.
5. Every first touch of code or data page-faults and is loaded by **Loader::loadPage**. The shell reads commands with **sc_read** and starts programs with **createprocess**.

11.3 Every chapter in the code

Chapter	Where in SWEB
1 processes	UserProcess (= a thread today), ProcessRegistry , createprocess ; no fork/exec yet (Team F)
2 threads	pthread.c stubs; Thread with kernel_stack_ and two register sets
3 races	a preemptive kernel: interrupts are on during system calls
4 locks	SpinLock (test-and-set + yield), Mutex (sleeps), ArchThreads::testSetLock , interrupts off in schedule()
5 condvars	Condition ; Scheduler::sleep/wake with the wait-until-sleeping loop
6 deadlocks	holding_lock_list_ , lock_waiting_on_ , doChecksBeforeWaiting ; no locks in interrupt handlers
7 syscalls	int \$0x80 , arch_syscallHandler , Syscall::syscallException , buffer + size > USER_BREAK checks
8 memory	ArchMemory::mapPage/unmapPage , PageManager , PageFaultHandler , Loader::loadPage , identity mapping
9 scheduling	round robin in Scheduler::schedule , one tick per quantum, yield = int \$65
10 threads	Thread states, kill → ToBeDestroyed → cleanup thread

11.4 What A1 changes

Phase 0 (all four, 1–4 Oct): split process from thread. A process owns the address space (**Loader/ArchMemory**), the file descriptors and the PID; a thread owns its stacks, registers, state and TID. Consequences to design together: how a thread finds its process (a pointer; keeping **Thread::loader_** as a non-owning shortcut saves many edits), who destroys the process and when (after its *last* thread, in the cleanup thread — never on a stack or in an address space that is about to disappear), how ids are handed out (today every **tid_** is 0), and what **ProcessRegistry** counts.

Team P. **pthread_create** (P1): the system call, argument checks, a new thread with its own user stack and a start wrapper. **pthread_exit** (P2): the return value survives the thread, the stack is freed, the process keeps running. **pthread_join** (P1): block until the target has finished, receive exactly the stored value; a detached thread cannot be joined. **pthread_cancel** (P2): end the target at a defined cancellation point.

Team F. `fork` with copy-on-write, `exec` — and, together with Team P, their behaviour in multi-threaded processes (`fork` duplicates only the calling thread; `exit` tears down every thread).

11.5 Testing

The course grades with your own tests and with the Tacos test system; neither replaces the other. Practical rules:

- Write the test program first (`userspace/tests/`); re-run CMake after adding one, and build twice if the new program is missing from the image.
- Test the edges, not only the happy path: bad user pointers, joining yourself or twice, main exiting first, a thread creating threads, many threads, create/join in a loop (leaks).
- Run every test many times: module 11's `swab_run.py -n 10` boots SWEB headless ten times and reports hangs (exit 1) and kernel panics (exit 2). Races appear on the tenth run.
- Use the debug categories (`debug.h`) instead of scattering `kprintf`; switch noisy ones off. For hard cases: `make qemu gdb + make run gdb`.
- Assertions are cheap documentation: `assert(lock.isHeldBy(currentThread))` at the top of a function that needs the lock.

11.6 Pitfalls that cost days

- `new/delete` with interrupts off or in a handler: the kernel-heap lock may be held by the interrupted thread.
- Sleeping while holding a lock the waker needs; `Condition::wait` asserts against extra held locks for a reason.
- Deleting a thread (or an address space) that is still running or still in use — defer to the cleanup thread.
- Trusting user pointers, or checking them with an overflowing sum.
- Busy-waiting (`while (!done) yield();`) instead of sleeping on a condition.
- Two threads faulting on the same missing page table at once: `mapPage` needs a lock once a process has several threads.
- Checking once, using later (TOCTOU) — in user memory and in kernel state: “is the thread still alive?” must be answered and acted on under the same lock.

Key points

- Know the map: `common/source/kernel`, `mm`, `arch/x86/64`, `userspace/libc`, `userspace/tests`.
- Boot path: `startup` → `ProcessRegistry` → `UserProcess` → lazy loading by page faults.
- A1 starts with the process/thread split — agree on ownership, lifetime, ids and locking as a group.
- Test the edges, many times, headless; read the debug output.